PHOTONICSAI | AI-DRIVEN SCIENCE

Homework 1

Complete worked solutions and reproducible experiments

Topics: Neural networks, parameter counting, softmax, attention, PINNs, agentic systems

Due date: 11 October 2026

Dataset: Silverbox / SNLS80mV

Code: https://github.com/Maks-Grigoryan/photonicsai-homework-1

This report gives every requested weight, bias, shape, proof, free-run metric, and agent trace. Numerical claims in Tasks 5–6 come from the included scripts, not from hand-entered estimates.

Reproducibility note. The official dataset is not committed to the repository. Place SNLS80mV.csv under data/SilverboxFiles/ and run the commands in the final section.

Task 1. Two networks a line cannot be

Both targets are piecewise linear on the requested domain, but neither is a single affine function. ReLU is useful because each hidden unit contributes one hinge. The constructions below use two hidden units, and two is minimal in both parts.

(a) Absolute value

Use one hinge for the positive half-line and a second hinge for the negative half-line:

$$h_1 = ReLU(x), \quad h_2 = ReLU(-x), \quad \hat{y} = h_1 + h_2.$$

In matrix form, the hidden layer and output layer are

$$W^{(1)} = [1; -1], \quad b^{(1)} = [0; 0], \quad \sigma^{(1)} = ReLU; \quad W^{(2)} = [1 \ 1], \quad b^{(2)} = 0, \quad \sigma^{(2)}(z)=z.$$

If $x \geq 0$, then $h_1=x$ and $h_2=0$. If $x<0$, then $h_1=0$ and $h_2=-x$. Therefore $\hat{y}=|x|$ for every real x.

x	$z_1=x$	h_1	$z_2=-x$	h_2	$\hat{y}$
−2	−2	0	2	2	2
−0.5	−0.5	0	0.5	0.5	0.5
0	0	0	0	0	0
0.5	0.5	0.5	−0.5	0	0.5
3	3	3	−3	0	3

Requested negative-input trace. At $x=-2$, unit 1 has pre-activation $z_1=1\cdot(-2)+0=-2$ and activation $h_1=ReLU(-2)=0$. Unit 2 has $z_2=(-1)\cdot(-2)+0=2$ and $h_2=ReLU(2)=2$. The identity output is $1\cdot0+1\cdot2+0=2$.

Minimality. With one hidden ReLU, a linear output has the form $A\cdot ReLU(wx+b)+c$. Its slope is zero on one side of its hinge and Aw on the other. The function $|x|$ instead has two nonzero slopes, −1 and +1, so one hidden unit cannot work. Thus the minimum is two.

(b) Exclusive OR

On the Boolean square, XOR is the absolute difference $|x_1 - x_2|$. Reuse the two-hinge construction:

$$h_1 = ReLU(x_1 - x_2), \quad h_2 = ReLU(x_2 - x_1), \quad \hat{y} = h_1 + h_2.$$

$$W^{(1)} = [[1,-1],[-1,1]], \quad b^{(1)}=[0,0], \quad \sigma^{(1)}=ReLU; \quad W^{(2)}=[1,1], \quad b^{(2)}=0, \quad \sigma^{(2)}(z)=z.$$

x_1	x_2	z_1	h_1	z_2	h_2	$\hat{y}$
0	0	0	0	0	0	0
0	1	−1	0	1	1	1
1	0	1	1	−1	0	1
1	1	0	0	0	0	0

Why one neuron is insufficient. The two positive XOR points, (1,0) and (0,1), are on opposite corners from the two negative points, (0,0) and (1,1); no single affine decision boundary can separate those two diagonal pairs. The two-ReLU network above therefore uses the minimum number of hidden units.

Task 2. Parameter count and layer shapes

For every convolution or pool I apply $H_{out}=\lfloor (H+2p-k)/s \rfloor+1$. A convolution contributes $k^2 C_{in} C_{out}+C_{out}$ parameters; a dense layer contributes $N_{in}N_{out}+N_{out}$. ReLU and max-pool have no trainable parameters.

$$\text{Input shape} = 224 \times 224 \times 3.$$

The spatial calculations are:

1. C1: $\lfloor (224+2\cdot 3-7)/2 \rfloor+1 = \lfloor 223/2 \rfloor+1 = 112$.
2. P1: $\lfloor (112-3)/2 \rfloor+1 = 55$.
3. C2: $\lfloor (55+2\cdot 2-5)/1 \rfloor+1 = 55$; P2 gives $\lfloor (55-3)/2 \rfloor+1 = 27$.
4. C3, C4, C5 each use k=3, s=1, p=1, so the side stays 27.
5. P3: $\lfloor (27-3)/2 \rfloor+1 = 13$; the flattened volume is $13\cdot 13\cdot 256 = 43{,}264$.

Layer	Output shape	Parameter calculation	Parameters
Input	224 × 224 × 3	—	0
C1 + ReLU	112 × 112 × 64	$7^2\cdot 3\cdot 64+64$	9,472
P1	55 × 55 × 64	—	0
C2 + ReLU	55 × 55 × 192	$5^2\cdot 64\cdot 192+192$	307,392
P2	27 × 27 × 192	—	0
C3 + ReLU	27 × 27 × 384	$3^2\cdot 192\cdot 384+384$	663,936
C4 + ReLU	27 × 27 × 256	$3^2\cdot 384\cdot 256+256$	884,992
C5 + ReLU	27 × 27 × 256	$3^2\cdot 256\cdot 256+256$	590,080
P3	13 × 13 × 256	—	0
Flatten	43,264	—	0
F6 + ReLU	512	$43{,}264\cdot 512+512$	22,151,680
F7 + ReLU	256	$512\cdot 256+256$	131,328
F8	10	$256\cdot 10+10$	2,570

Summing only the trainable layers:

$$9{,}472 + 307{,}392 + 663{,}936 + 884{,}992 + 590{,}080 + 22{,}151{,}680 + 131{,}328 + 2{,}570 = 24{,}741{,}450.$$

Final answer. The network has 24,741,450 trainable parameters. The dominant term is F6 because flattening produces 43,264 inputs before the 512-unit dense layer.

Task 3. Softmax is a probability mass function

(a) Positivity and normalization

Let $z \in \mathbb{R}^K$ and define $s_i(z)=\exp(z_i)/\Sigma_{j=1}^{K} \exp(z_j)$. For every real z_i, $\exp(z_i)>0$. The denominator is a finite sum of strictly positive terms, so it is also strictly positive. Hence $s_i(z)>0$ for every class i.

For normalization, the denominator is the same for every component:

$$\Sigma_{i=1}^{K} s_i(z) = \Sigma_{i=1}^{K} exp(z_i) / \Sigma_{j=1}^{K} exp(z_j) = (\Sigma_{i=1}^{K} exp(z_i)) / (\Sigma_{j=1}^{K} exp(z_j)) = 1.$$

Therefore the K softmax outputs are nonnegative (in fact strictly positive) and sum to one, which is exactly the definition of a probability mass function on the finite set {1,…,K}.

(b) Why it is not a probability density

The normalization just proved is a finite sum over class labels. Softmax returns one number for each discrete outcome $i \in \{1,\ldots,K\}$. A probability density instead is a function of a continuous outcome, and its normalization has the form $\int f(x)dx=1$ over that outcome space. Softmax has no continuous outcome variable and no integral. Thus it is a discrete probability mass function, not a continuous probability density.

Task 4. A dot product does not know the seat

(a) The score is a dot product

Write q_i for row i of Q and k_j for row j of K. Matrix multiplication gives

$$(QK^T)_{ij} = \Sigma_r\, Q_{ir}K_{jr} = q_i{}^T k_j.$$

After reordering rows, X'=PX. Because the projection acts on features rather than seats, Q'=PXWQ=PQ and K'=PXWK=PK. With the convention $P_{i,\pi(i)}=1$, row i of Q' is row π(i) of Q. Therefore

$$(q'_i)^T k'_j = q\pi(i)^T k\pi(j).$$

If one token moves from seat 1 to seat 3, every dot product between its query/key vector and the other token vectors keeps the same value; only the row and column in which that value is stored change.

Two-token check. Let u=(1,0), v=(0,1), and WQ=WK=I. With seats [u,v], the score table is:

query / key	u (seat 1)	v (seat 2)
u (seat 1)	u·u = 1	u·v = 0
v (seat 2)	v·u = 0	v·v = 1

After swapping the seats to [v,u], the table is:

query / key	v (seat 1)	u (seat 2)
v (seat 1)	v·v = 1	v·u = 0
u (seat 2)	u·v = 0	u·u = 1

Numerically both matrices are [[1,0],[0,1]], but the labeled entries moved: u·u moved from (1,1) to (2,2), v·v moved from (2,2) to (1,1), and the two cross terms exchanged (1,2) and (2,1).

(b) The whole layer follows the tokens

Let $S=QK^T/\sqrt{d_k}$. Under the row permutation:

$$S' = (PQ)(PK)^T/\sqrt{d_k} = P(QK^T)P^T/\sqrt{d_k} = PSP^T.$$

A left multiplication by P reorders rows and a right multiplication by P^T reorders the entries within each row. Row-wise softmax commutes with both reorderings, so if A=softmax(S), then

$$softmax(PSP^T) = P\,A\,P^T.$$

The values also follow the tokens: V'=PXWV=PV. Consequently,

$$Att(PX) = (PAP^T)(PV) = PA(P^TP)V = PAV = P\,Att(X).$$

Thus the output row associated with a token moves to the token's new seat, but its value is unchanged apart from that row permutation.

For u and v, set WV=I and omit $1/\sqrt{d_k}$. Let p=e/(e+1)≈0.731059 and q=1/(e+1)≈0.268941. Since the score matrix is I, row-wise softmax is [[p,q],[q,p]]. Therefore

$$Att(X) = [[p,q],[q,p]].$$

After the swap, V'=[v;u]. The first output is pv+qu=(q,p), and the second is qv+pu=(p,q):

$$Att(PX) = [[q,p],[p,q]] = P\,Att(X).$$

(c) Position has to enter before the dot product

With seat encodings $E=[e_1;e_2]$, the unswapped input is X+E. After swapping only the tokens while encodings remain attached to seats, the actual input is PX+E, whose rows are $[x_2+e_1;\ x_1+e_2]$. In contrast, permuting the already encoded input gives

$$P(X+E) = PX+PE = [x_2+e_2;\ x_1+e_1].$$

If $e_1 \neq e_2$ then PE≠E, hence PX+E≠P(X+E). The vectors entering the query/key projections are now different, so their dot products can reveal the seats. Without E, any stack made from permutation-equivariant attention and the same row-wise function still obeys F(PX)=PF(X); order becomes visible only because position was inserted before the dot product.

Task 5. PINN for the Silverbox oscillator

Data and split

I used the official SNLS80mV record and took the exact Python slice [42650:46746]. The excerpt means were $\bar{u}$=0.0066574164 V and $\bar{y}$=0.0013167126 V; both were subtracted before any derivative, training, or evaluation. With fs=610.35 Hz, Δt=0.0016384042 s. The first 3072 samples last 5.03318 s and form the training record; the final 1024 last 1.67773 s and form the test record.

The zero-prediction test baseline is RMS(ytest)=0.06049180 V. This is useful as a sanity check: a learned model should be well below that value.

State equation and neural acceleration

$$\ddot{y} + a\,\dot{y} + b\,y + c\,y^3 = g\,u(t), \quad \dot{y}=v, \quad \dot{v}=\alpha\theta(y,v,u).$$

The kept acceleration model is a fully connected 3→64→64→32→1 MLP. Hidden layers use ReLU and the output is linear. It has 6,529 trainable weights and biases. I normalize the three inputs by training scales σy=0.0530193 V, σv=21.3602 V/s, and σu=0.0219567 V, then multiply the normalized network output by σα=10,595.43 V/s².

The physics-informed initialization uses a seven-point, fourth-order Savitzky–Golay finite-difference filter on the training record only. A positive-coefficient short-rollout fit gives a=40.7887 s^{-1}, b=184,965.2 s^{-2}, c=751,358 in the corresponding voltage units, and g=193,864.0 s^{-2}. These coefficients define smooth acceleration targets for pretraining; the reported rollout is produced by the MLP, not by substituting this closed-form force at evaluation time.

RK4 step

For state s=(y,v), define f(s,u)=(v,αθ(y,v,u)). Between measured inputs uk and uk+1 I linearly use their midpoint um=(uk+uk+1)/2 and compute

$$k_1=f(sk,uk),\ k_2=f(sk+\Delta t\,k_1/2,um),\ k_3=f(sk+\Delta t\,k_2/2,um),\ k_4=f(sk+\Delta t\,k_3,uk+1),$$

$$sk+1 = sk + (\Delta t/6)(k_1+2k_2+2k_3+k_4).$$

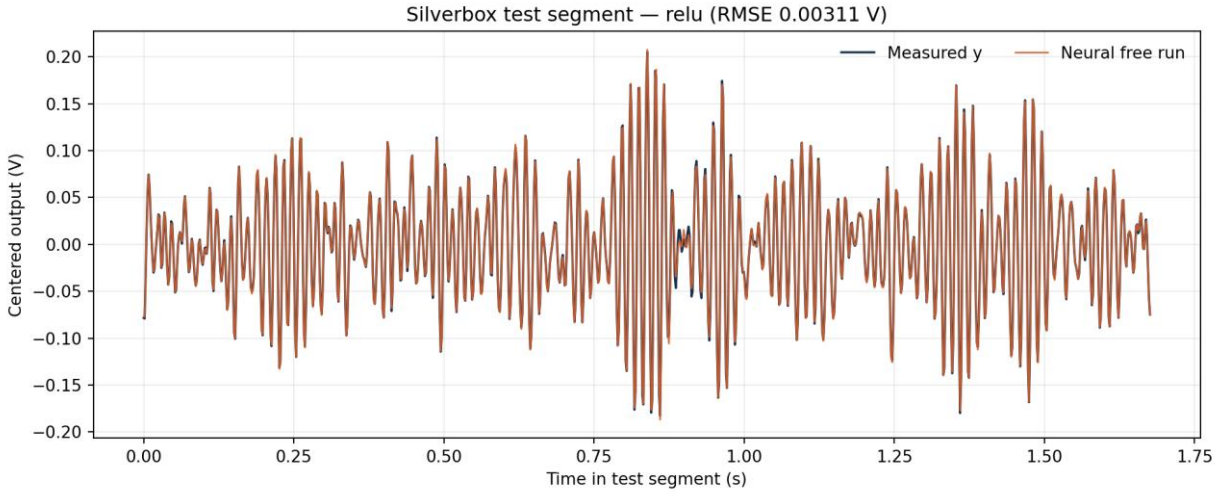
Training procedure and hyperparameters

Step	Action
1	Fit the physics teacher only on the first 3072 samples and pretrain $\alpha\theta$ on the resulting acceleration field.
2	Fine-tune through differentiable RK4 rollouts. A batch contains 24 randomly chosen training starts, each 96 steps long; later rollout steps receive slightly larger loss weight to penalize drift.
3	For the kept ReLU model, use AdamW at 1.5×10^{-3} with cosine decay for 2,000 pretraining steps, then Adam at 1.8×10^{-4} for 320 rollout steps. Clip the gradient norm and use seed 17.
4	For the required free run, initialize y0 from the measurement and $v0=(-3y0+4y1-y2)/(2\Delta t)=-13.626674$ V/s. Never reset the state at sample 3072 and never use test y as an input.

Trial	Architecture	Act.	Initial LR	Steps	Train RMSE (V)	Test RMSE (V)
small_tanh	3→32→32→1	tanh	0.002	2400 + 450	0.00156560	0.00877505
deep_tanh	3→64→64→32→1	tanh	0.0015	1800 + 320	0.00124866	0.00874710
relu	3→64→64→32→1	relu	0.0015	2000 + 320	0.00106542	0.00311170

Kept version. The ReLU model has the smallest test error: train free-run RMSE 0.00106542 V and test free-run RMSE 0.00311170 V. The test error is about 19.4 times smaller than the zero baseline. The small train/test gap also indicates that the state did not simply fit the first segment and then diverge.

Figure 1. Measured and predicted centered output on the 1024-sample test segment.

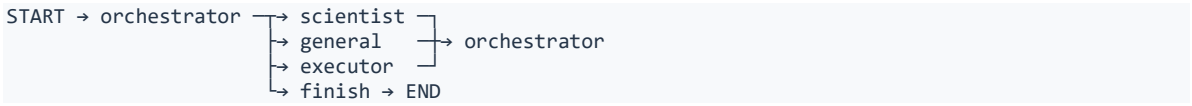


Code repository: https://github.com/Maks-Grigoryan/photonicsai-homework-1

Task 6. Agentic system

Graph and shared state

The implementation is a LangGraph StateGraph with exactly four nodes. The only shared communication channel is an append-only dialogue; each item stores the speaker role and text. The state also carries decision_count, next_role, the current instruction, and the run directory.



Node	Responsibility and constraint
Orchestrator	Model-scored JSON decision: next_role, instruction, reason. No theory and no coding tool.
Scientist	Duffing equation, neural state model, training/free-run plan, and numbered implementation steps. No coding tool and no invented scores.
General	Adjacent definitions such as one record and the meaning of free-run RMSE. No fitting plan and no coding tool.
Executor	The only node with the write-and-run tool. Writes a new Python file, runs it with a 300 s timeout, and returns exit code/stdout/stderr.

Model-based routing and safety conditions

The router uses the local instruction model Qwen2.5-0.5B-Instruct. For the current dialogue, the model scores complete candidate JSON decisions by mean conditional token log-probability; the highest-scoring state-valid decision wins. This is a model call over the dialogue, not a keyword table and not a fixed specialist order.

Graph guards encode the assignment rather than domain routing: executor is unavailable until a scientist plan exists; finish is unavailable until an executor turn contains exit_code=0 and a printed test RMSE; and the graph stops after at most eight decisions. After any specialist turn, the only ordinary edge returns to the orchestrator so the new message is visible.

The coding tool writes the full validated Task 5 implementation into a fresh run directory as task5_generated.py. It then executes that file, captures standard output and standard error, and enforces the five-minute limit. The executor additionally checks that its architecture matches the latest scientist profile.

Validation on Task 5

Turn	Speaker	Action	Visible result / reason
1	Orchestrator	Selected general	The conceptual side question was still unanswered.
2	General	Defined record and RMSE	One sample is (u[k],y[k]); free-run RMSE includes accumulated drift without y feedback.
3	Orchestrator	Selected scientist	A complete physical and coding plan was still needed.
4	Scientist	Proposed ReLU PINN	3→64→64→32→1, training-only derivatives, RK4, continuous free run.
5	Orchestrator	Selected executor	A script had to produce a measured RMSE.
6	Executor	Wrote and ran code	exit_code=0; train 0.00106542 V; test 0.00311170 V.
7	Orchestrator	Selected finish	A successful executor RMSE was present.

The decisive executor output was:

```
exit_code=0
RESULT relu: train_free_run_RMSE=0.00106542 V, test_free_run_RMSE=0.00311170 V
BEST relu: test_free_run_RMSE=0.00311170 V
```

The graph used four orchestrator decisions (general, scientist, executor, finish), so it terminated well before the eight-decision cap. The full role-labelled transcript is saved under task6_agentic_system/runs/validation_run_5/transcript.md.

Code repository: https://github.com/Maks-Grigoryan/photonicsai-homework-1

Reproducibility and files

The repository separates the numerical model from the agentic wrapper. The important files are:

Path	Purpose
task5_pinn/train_silverbox_pinn.py	Loads/centers data, trains three MLPs, performs RK4 free runs, saves metrics and Figure 1.
task5_pinn/results/metrics.json	Machine-readable architecture settings and final train/test RMSE values.
task6_agentic_system/agentic_task5.py	Four-node StateGraph, local model router, role nodes, coding tool, and stop guards.
task6_agentic_system/runs/validation_run_5/transcript.md	Complete role-labelled validation dialogue and executor output.

Task 5 command:

```
py task5_pinn/train_silverbox_pinn.py
```

Task 6 command:

```
.\.venv\Scripts\python.exe task6_agentic_system\agentic_task5.py
```

Repository: https://github.com/Maks-Grigoryan/photonicsai-homework-1

Sources

Silverbox benchmark and official data download: nonlinearbenchmark.org/benchmarks/silverbox

T. Wigren and J. Schoukens, “Three free data sets for development and benchmarking in nonlinear system identification,” European Control Conference, 2013, pp. 2933–2938, doi:10.23919/ECC.2013.6669201.